

你可以把这两份文档看作是“大脑”与“肉体”的关系：

- **SOP (PDF 1)** 偏向于物理世界数字化与系统工程，它定义了如何搭建高保真的虚拟沙盒、如何设置物理材质(摩擦力)，以及如何做后端的并发测试。
- **算法总结 (PDF 2)** 是这个系统的大脑中枢，它定义了智能体如何听懂用户的错别字，以及如何在复杂的网格中找到“最安全”的路径。

将你的想法与SOP结合，具体的执行路径应该是这样的：

## 第一步：脱机压力测试(你的1000条数据模拟)

这属于 **SOP** 阶段三之前的“算法单元测试”。

在把算法放进三维空间之前，必须先证明它的“大脑”是好用的。完全赞同你先跑 1000 条数据。

- **做法**: 纯代码层面运行。写一个 Python 脚本，生成 1000 个包含拼音残缺、生僻字错误的测试请求(也就是验证算法中提到的能否处理“海田烫够”等极端情况)。
- **产出**: 生成一套准确率与容错率的数据分析图(混淆矩阵、处理延迟时长等)。
- **价值**: 用硬核数据证明你的“多级容错意图识别”机制 在文本/NLP层面的有效性。

## 第二步：Unity A/B 对照测试(SOP阶段三的“灵魂升级”)

这是对 SOP 中“阶段三：蒙特卡洛大规模测试”的具象化与升级。

SOP 里提到要在场景中实例化 50-100 个虚拟代理进行“随机巡逻与避障测试”。如果只是随机乱走，价值并不大。引入你的 **A/B** 组对照测试，整个测试就有了极强的实验性和学术价值：

- **A组(对照组/无算法)**: 使用传统的 NavMesh 或 Dijkstra 最短路径。它们会无视你在 SOP 阶段二中辛苦配置的物理材质(摩擦力)和地形坡度。
- **B组(实验组/你的算法)**: 接入你的加权空间图谱算法，针对视障群体调用 `get_safest_route`，动态规避危险节点。
- **产出**: A 组的 Agent 会在陡坡或特定材质上频繁卡死或触发碰撞，产生大量的风险数据推送给后台；而 B 组的 Agent 能够平稳导航。
- **价值**: 这直接呼应了 SOP 阶段五的“演示视频(Demo Reel)制作”。通过左右分屏对比，你能以极强的视觉冲击力证明你的算法确实实现了“事后治理”向“事前预防”的转化。

---

## 总结

你的思路非常清晰，逻辑严密，把原本有些宏大的 SOP 落地成了可执行的科学实验。严格来说，先跑 **1000 条 Python** 假数据验证 **NLP** 容错，再去 **Unity** 里跑 **A/B** 组验证空间加权路径，就是目前最完美的 workflow。